

Dinkel: State-Aware and Granular Framework for Validating Graph Databases

Celine Wüst Zu-Ming Jiang Zhendong Su

ETH Zürich, Switzerland

July 20, 2026

- ① Background & Motivation
- ② Approach
- ③ Evaluation
- ④ Conclusion

Graph Databases and the Cypher Language

- **GDBMSs** power modern data-driven applications (75% of Fortune 100 use Neo4j).
- Data model: *labeled property graph* — nodes + relationships (edges), both can carry *labels* and *properties*.

Graph Databases and the Cypher Language

- **GDBMSs** power modern data-driven applications (75% of Fortune 100 use Neo4j).
- Data model: *labeled property graph* — nodes + relationships (edges), both can carry *labels* and *properties*.
- **Cypher**: the most widely adopted graph query language.
- Unlike SQL, Cypher does *not* separate DDL / DML / DQL: a single query can **create, read, and modify** data.
- Clauses are executed *sequentially*; state changes made by one clause are **visible to subsequent clauses**.

Graph Databases and the Cypher Language

- **GDBMSs** power modern data-driven applications (75% of Fortune 100 use Neo4j).
- Data model: *labeled property graph* — nodes + relationships (edges), both can carry *labels* and *properties*.
- **Cypher**: the most widely adopted graph query language.
- Unlike SQL, Cypher does *not* separate DDL / DML / DQL: a single query can **create, read, and modify** data.
- Clauses are executed *sequentially*; state changes made by one clause are **visible to subsequent clauses**.

<i>query</i>	::=	<i>clause</i> [<i>query</i>]
<i>clause</i>	::=	<i>reading_clause</i> <i>writing_clause</i> <i>reading/writing_clause</i> <i>projecting_clause</i> ...
<i>reading_clause</i>	::=	['OPTIONAL'] 'MATCH' <i>pattern</i> ['WHERE' <i>expression</i>]
<i>pattern</i>	::=	<i>node</i> [<i>relationship pattern</i>]
<i>node</i>	::=	'(' <i>label</i> * <i>properties?</i>)'
<i>relationship</i>	::=	'<-' <i>label properties?</i> ']-' '-[' <i>label properties?</i> ']->'
<i>label</i>	::=	'.' <i>identifier</i>
<i>properties</i>	::=	'{' <i>identifier</i> : <i>expression</i> [, <i>identifier</i> : <i>expression</i>] '}'
<i>expression</i>	::=	<i>identifier</i> <i>constant</i> <i>operation</i> <i>function</i> ...
<i>writing_clause</i>	::=	<i>create_clause</i> <i>delete_clause</i> <i>set_clause</i> <i>remove_clause</i> <i>foreach_clause</i> ...
<i>reading/writing_clause</i>	::=	<i>merge_clause</i> <i>call_clause</i> ...
<i>projecting_clause</i>	::=	<i>return_clause</i> <i>with_clause</i> <i>unwind_clause</i> ...

Challenge 1: Generating Complex and Valid Queries

Many deep bugs are only triggered by **complex queries**: various clause combinations + complicated data dependencies.

Challenge 1: Generating Complex and Valid Queries

Many deep bugs are only triggered by **complex queries**: various clause combinations + complicated data dependencies.

Why is this hard for Cypher?

- Clauses *change the graph state during execution*; the effects are visible to later clauses.
- Variables have *types and scopes* (e.g., variables die after UNION, subquery variables are local).

Challenge 1: Generating Complex and Valid Queries

Many deep bugs are only triggered by **complex queries**: various clause combinations + complicated data dependencies.

Why is this hard for Cypher?

- Clauses *change the graph state during execution*; the effects are visible to later clauses.
- Variables have *types and scopes* (e.g., variables die after UNION, subquery variables are local).

Dilemma of existing approaches (no systematic state modeling):

- Ignore state changes $\Rightarrow$ queries are too *simple*, deep GDBMS logic (planner / executor) is never exercised.
- Handle states incorrectly $\Rightarrow$ queries are *invalid* (reference nonexistent or out-of-scope items), rejected at parsing.

Challenge 2: Lack of a General Test Oracle

- **Logic bugs** silently produce wrong results — need a *test oracle* to validate query results.
- Existing oracles transform a query and compare results, but all work at the **query level**: they must understand the whole-query semantics.

Challenge 2: Lack of a General Test Oracle

- **Logic bugs** silently produce wrong results — need a *test oracle* to validate query results.
- Existing oracles transform a query and compare results, but all work at the **query level**: they must understand the whole-query semantics.

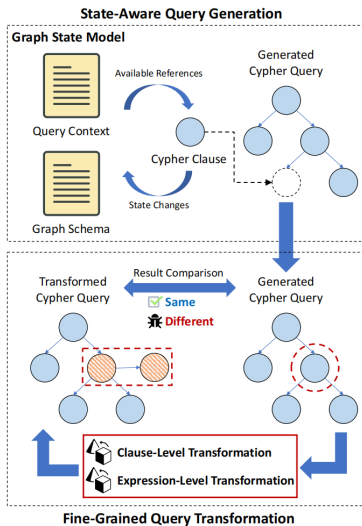
Consequence: only queries matching specific patterns can be validated.

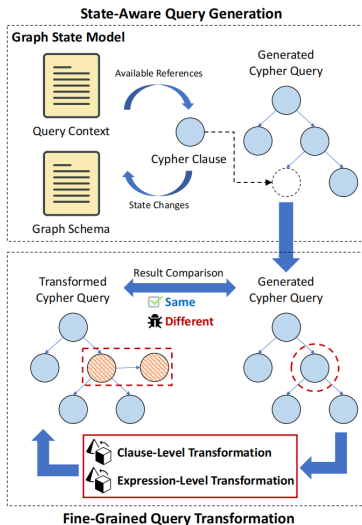
- GraphGenie: only *single* MATCH + RETURN queries.
- GRev: predicate modeling only for MATCH statements.
- MERGE, UNWIND, FOREACH, UNION, ... remain **unsupported and untested** $\Rightarrow$ many logic bugs missed.

```
// Original: 2 rows   
RETURN 0 AS n1  
UNION CALL {  
  FOREACH (n2 IN [] |  
    FOREACH (n3 IN [] |  
      MERGE (:A {u:0})))  
} MATCH (y)  
RETURN y AS n1  
UNION CREATE ()  
RETURN 0 AS n1
```

```
// Transformed: 1 row   
FOREACH (n0 IN [] |  
  FOREACH (x IN [] |  
    SET x = {}))  
RETURN 0 AS n1  
UNION CALL {  
  FOREACH (n2 IN [] |  
    FOREACH (n3 IN [] |  
      MERGE (:A {u:0})))  
} MATCH (y)  
RETURN y AS n1  
UNION CREATE ()  
RETURN 0 AS n1
```

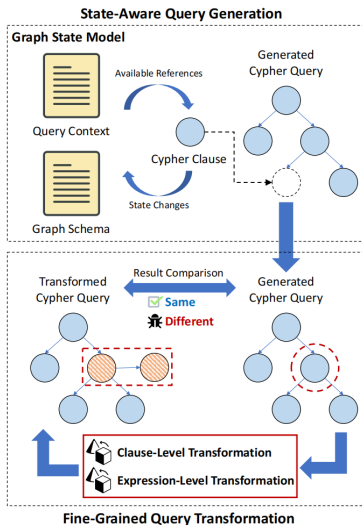
Dinkel: Key Ideas





Solution 1: State-Aware Query Generation

Model graph states as *query context* + *graph schema*; generate queries **clause by clause**, updating states on the fly.



Solution 1: State-Aware Query Generation

Model graph states as *query context* + *graph schema*; generate queries **clause by clause**, updating states on the fly.

Solution 2: Fine-Grained Query Transformation

Semantic-preserving transformations on **clauses** and **expressions** (inspired by EET for SQL, generalized to state-changing clauses) — applicable to *arbitrary* queries; result difference $\Rightarrow$ logic bug.

Query Context (qc)

- Temporary **variables** declared in the query, with their *types* and *scopes*.
- Concrete values (0) or aliases of nodes / relationships (x, y).
- Only referable *after definition, within scope*; cleared when out of scope (e.g., after UNION).

Graph State Modeling: Two Abstractions

Query Context (qc)

- Temporary **variables** declared in the query, with their *types* and *scopes*.
- Concrete values (0) or aliases of nodes / relationships (x, y).
- Only referable *after definition, within scope*; cleared when out of scope (e.g., after UNION).

Graph Schema (gs)

- **Labels** and **properties** of the stored graph (e.g., label :A, property u).
- Changes are *permanent* (not scoped): CREATE adds, REMOVE removes.
- May legally reference nonexistent labels/properties (just match nothing).

Graph State Modeling: Two Abstractions

Query Context (qc)

- Temporary **variables** declared in the query, with their *types* and *scopes*.
- Concrete values (0) or aliases of nodes / relationships (x, y).
- Only referable *after definition, within scope*; cleared when out of scope (e.g., after UNION).

Graph Schema (gs)

- **Labels** and **properties** of the stored graph (e.g., label :A, property u).
- Changes are *permanent* (not scoped): CREATE adds, REMOVE removes.
- May legally reference nonexistent labels/properties (just match nothing).

Key insight

Clauses are the **basic units** operating on graphs; qc and gs are updated *only* when a clause is invoked or a clause context is exited.

State-Aware Query Generation (Algorithm 1)

Algorithm 1: State-Aware Query Generation

Output: *query*

```
1 Function GenQuery():
2   query  $\leftarrow$  EmptyQuery();
3   qc  $\leftarrow$  {}; // query context
4   gs  $\leftarrow$  {}; // graph schema
5   do
6     clause, qc, gs  $\leftarrow$  GenClause(qc, gs, ANY);
7     AppendClause(query, clause);
8   while rand() < P and Length(query) <  $L_{max}$ ;
9   return query;
10 Function GenClause(qc, gs, type):
11   if type = ANY then
12     type = RandClauseType();
13   // initialize the clause accordingly
14   clause, qc, gs  $\leftarrow$  RandInitClause(qc, gs, type);
15   foreach subclause, subclause_type in clause do
16     subclause, qc, gs  $\leftarrow$  GenClause(qc, gs,
17     subclause_type);
18   // clean out-of-scope query context
19   CleanQC(qc);
20   return clause, qc, gs;
```

State-Aware Query Generation (Algorithm 1)

Algorithm 1: State-Aware Query Generation

Output: *query*

```
1 Function GenQuery():  
2   query  $\leftarrow$  EmptyQuery();  
3   qc  $\leftarrow$  {}; // query context  
4   gs  $\leftarrow$  {}; // graph schema  
5   do  
6     clause, qc, gs  $\leftarrow$  GenClause(qc, gs, ANY);  
7     AppendClause(query, clause);  
8   while rand() < P and Length(query) <  $L_{max}$ ;  
9   return query;  
10 Function GenClause(qc, gs, type):  
11   if type = ANY then  
12     type = RandClauseType();  
13     // initialize the clause accordingly  
14     clause, qc, gs  $\leftarrow$  RandInitClause(qc, gs, type);  
15     foreach subclause, subclause_type in clause do  
16       subclause, qc, gs  $\leftarrow$  GenClause(qc, gs,  
17         subclause_type);  
18     // clean out-of-scope query context  
19     CleanQC(qc);  
20   return clause, qc, gs;
```

① Start from an *empty* query, $qc = \{\}$, $gs = \{\}$.

State-Aware Query Generation (Algorithm 1)

Algorithm 1: State-Aware Query Generation

Output: *query*

```
1 Function GenQuery():
2   query  $\leftarrow$  EmptyQuery();
3   qc  $\leftarrow$  {}; // query context
4   gs  $\leftarrow$  {}; // graph schema
5   do
6     | clause, qc, gs  $\leftarrow$  GenClause(qc, gs, ANY);
7     | AppendClause(query, clause);
8   while rand() < P and Length(query) <  $L_{max}$ ;
9   return query;
10 Function GenClause(qc, gs, type):
11   if type = ANY then
12     | type = RandClauseType();
13   // initialize the clause accordingly
14   clause, qc, gs  $\leftarrow$  RandInitClause(qc, gs, type);
15   foreach subclause, subclause_type in clause do
16     | subclause, qc, gs  $\leftarrow$  GenClause(qc, gs,
17     | subclause_type);
18   // clean out-of-scope query context
19   CleanQC(qc);
20   return clause, qc, gs;
```

- ① Start from an *empty* query, $qc = \{\}$, $gs = \{\}$.
- ② Repeatedly generate a new clause:
 - randomly pick a clause type;
 - build the clause referencing **only accessible elements** in current *qc/gs*;
 - **recursively** generate subclauses (CALL $\{\}$, FOREACH body, ...);
 - **update** *qc/gs* immediately; clean out-of-scope variables.

State-Aware Query Generation (Algorithm 1)

Algorithm 1: State-Aware Query Generation

Output: *query*

```
1 Function GenQuery():
2   query  $\leftarrow$  EmptyQuery();
3   qc  $\leftarrow$  {}; // query context
4   gs  $\leftarrow$  {}; // graph schema
5   do
6     | clause, qc, gs  $\leftarrow$  GenClause(qc, gs, ANY);
7     | AppendClause(query, clause);
8   while rand() < P and Length(query) <  $L_{max}$ ;
9   return query;
10 Function GenClause(qc, gs, type):
11   if type = ANY then
12     | type = RandClauseType();
13     // initialize the clause accordingly
14   clause, qc, gs  $\leftarrow$  RandInitClause(qc, gs, type);
15   foreach subclause, subclause_type in clause do
16     | subclause, qc, gs  $\leftarrow$  GenClause(qc, gs,
17     | subclause_type);
18   // clean out-of-scope query context
19   CleanQC(qc);
20   return clause, qc, gs;
```

- ① Start from an *empty* query, $qc = \{\}$, $gs = \{\}$.
- ② Repeatedly generate a new clause:
 - randomly pick a clause type;
 - build the clause referencing **only accessible elements** in current *qc/gs*;
 - **recursively** generate subclauses (CALL $\{\}$, FOREACH body, ...);
 - **update** *qc/gs* immediately; clean out-of-scope variables.
- ③ Continue with probability P until length limit L_{max}

State-Aware Query Generation (Algorithm 1)

Algorithm 1: State-Aware Query Generation

Output: *query*

```
1 Function GenQuery():
2   query  $\leftarrow$  EmptyQuery();
3   qc  $\leftarrow$  {}; // query context
4   gs  $\leftarrow$  {}; // graph schema
5   do
6     | clause, qc, gs  $\leftarrow$  GenClause(qc, gs, ANY);
7     | AppendClause(query, clause);
8   while rand() < P and Length(query) <  $L_{max}$ ;
9   return query;
10 Function GenClause(qc, gs, type):
11   if type = ANY then
12     | type = RandClauseType();
13   // initialize the clause accordingly
14   clause, qc, gs  $\leftarrow$  RandInitClause(qc, gs, type);
15   foreach subclause, subclause_type in clause do
16     | subclause, qc, gs  $\leftarrow$  GenClause(qc, gs,
17       | subclause_type);
18   // clean out-of-scope query context
19   CleanQC(qc);
20   return clause, qc, gs;
```

- ① Start from an *empty* query, $qc = \{\}$, $gs = \{\}$.
 - ② Repeatedly generate a new clause:
 - randomly pick a clause type;
 - build the clause referencing **only accessible elements** in current *qc/gs*;
 - **recursively** generate subclauses (CALL $\{\}$, FOREACH body, ...);
 - **update** *qc/gs* immediately; clean out-of-scope variables.
 - ③ Continue with probability P until length limit L_{max}
- $\Rightarrow$ complex data dependencies **and** high validity at the same time.

Generation Walkthrough (the Query of Figure 2)

```
RETURN 0 AS n1
UNION CALL {
  FOREACH (n2 IN [] |
    FOREACH (n3 IN [] |
      MERGE (:A {u:0})))
} MATCH (y)
RETURN y AS n1
UNION CREATE ()
RETURN 0 AS n1
```

① empty query, $qc = \{\}$, $gs = \{\}$

① (empty)

② RETURN 0 AS n1

③ RETURN 0 AS n1 UNION

④ RETURN 0 AS n1 UNION CALL {}

⑤ RETURN 0 AS n1 UNION CALL {FOREACH (...)}

⑥ RETURN 0 AS n1 UNION CALL {FOREACH (...)}
MATCH (y) RETURN y AS n1

⑦ RETURN 0 AS n1 UNION CALL {FOREACH (...)}
MATCH (y) RETURN y AS n1
UNION CREATE () RETURN 0 AS n1

Generation Walkthrough (the Query of Figure 2)

```
RETURN 0 AS n1
UNION CALL {
  FOREACH (n2 IN [] |
    FOREACH (n3 IN [] |
      MERGE (:A {u:0})))
} MATCH (y)
RETURN y AS n1
UNION CREATE ()
RETURN 0 AS n1
```

- ① empty query, $qc = \{\}$, $gs = \{\}$
- ② $\text{RETURN } 0 \text{ AS } n1 \Rightarrow qc = \{n1\}$

- 
- ① (empty)
 - ② `RETURN 0 AS n1`
 - ③ `RETURN 0 AS n1 UNION`
 - ④ `RETURN 0 AS n1 UNION CALL {}`
 - ⑤ `RETURN 0 AS n1 UNION CALL {FOREACH (...)}`
 - ⑥ `RETURN 0 AS n1 UNION CALL {FOREACH (...)}`
`MATCH (y) RETURN y AS n1`
 - ⑦ `RETURN 0 AS n1 UNION CALL {FOREACH (...)}`
`MATCH (y) RETURN y AS n1`
`UNION CREATE () RETURN 0 AS n1`

Generation Walkthrough (the Query of Figure 2)

```
RETURN 0 AS n1
UNION CALL {
  FOREACH (n2 IN [] |
    FOREACH (n3 IN [] |
      MERGE (:A {u:0})))
} MATCH (y)
RETURN y AS n1
UNION CREATE ()
RETURN 0 AS n1
```

- ① empty query, $qc = \{\}$, $gs = \{\}$
- ② `RETURN 0 AS n1` $\Rightarrow qc = \{n1\}$
- ③ `UNION` $\Rightarrow$ qc **cleared**

- 
- ① (empty)
 - ② `RETURN 0 AS n1`
 - ③ `RETURN 0 AS n1 UNION`
 - ④ `RETURN 0 AS n1 UNION CALL {}`
 - ⑤ `RETURN 0 AS n1 UNION CALL {FOREACH (...)}`
 - ⑥ `RETURN 0 AS n1 UNION CALL {FOREACH (...)}`
`MATCH (y) RETURN y AS n1`
 - ⑦ `RETURN 0 AS n1 UNION CALL {FOREACH (...)}`
`MATCH (y) RETURN y AS n1`
`UNION CREATE () RETURN 0 AS n1`

Generation Walkthrough (the Query of Figure 2)

```
RETURN 0 AS n1
UNION CALL {
  FOREACH (n2 IN [] |
    FOREACH (n3 IN [] |
      MERGE (:A {u:0})))
} MATCH (y)
RETURN y AS n1
UNION CREATE ()
RETURN 0 AS n1
```

- ① empty query, $qc = \{\}$, $gs = \{\}$
- ② `RETURN 0 AS n1` $\Rightarrow qc = \{n1\}$
- ③ `UNION` $\Rightarrow$ qc **cleared**
- ④ `CALL` with recursive subclauses

- 
- ① (empty)
 - ② `RETURN 0 AS n1`
 - ③ `RETURN 0 AS n1 UNION`
 - ④ `RETURN 0 AS n1 UNION CALL {}`
 - ⑤ `RETURN 0 AS n1 UNION CALL {FOREACH (...)}`
 - ⑥ `RETURN 0 AS n1 UNION CALL {FOREACH (...)}`
`MATCH (y) RETURN y AS n1`
 - ⑦ `RETURN 0 AS n1 UNION CALL {FOREACH (...)}`
`MATCH (y) RETURN y AS n1`
`UNION CREATE () RETURN 0 AS n1`

Generation Walkthrough (the Query of Figure 2)

```
RETURN 0 AS n1
UNION CALL {
  FOREACH (n2 IN [] |
    FOREACH (n3 IN [] |
      MERGE (:A {u:0})))
} MATCH (y)
RETURN y AS n1
UNION CREATE ()
RETURN 0 AS n1
```

- ① empty query, $qc = \{\}$, $gs = \{\}$
- ② `RETURN 0 AS n1` $\Rightarrow qc = \{n1\}$
- ③ `UNION` $\Rightarrow qc$ **cleared**
- ④ `CALL` with recursive subclauses
- ⑤ two `FOREACH` define $n2, n3$; `MERGE` adds label A , property u to gs ; after `FOREACH` ends, $n2, n3$ removed (out of scope)



- ① (empty)
- ② `RETURN 0 AS n1`
- ③ `RETURN 0 AS n1 UNION`
- ④ `RETURN 0 AS n1 UNION CALL {}`
- ⑤ `RETURN 0 AS n1 UNION CALL {FOREACH (...)}`
- ⑥ `RETURN 0 AS n1 UNION CALL {FOREACH (...)}`
`MATCH (y) RETURN y AS n1`
- ⑦ `RETURN 0 AS n1 UNION CALL {FOREACH (...)}`
`MATCH (y) RETURN y AS n1`
`UNION CREATE () RETURN 0 AS n1`

Generation Walkthrough (the Query of Figure 2)

```
RETURN 0 AS n1
UNION CALL {
  FOREACH (n2 IN [] |
    FOREACH (n3 IN [] |
      MERGE (:A {u:0})))
} MATCH (y)
RETURN y AS n1
UNION CREATE ()
RETURN 0 AS n1
```



① (empty)
② RETURN 0 AS n1
③ RETURN 0 AS n1 UNION
④ RETURN 0 AS n1 UNION CALL {}
⑤ RETURN 0 AS n1 UNION CALL {FOREACH (...)}
⑥ RETURN 0 AS n1 UNION CALL {FOREACH (...)}
MATCH (y) RETURN y AS n1
⑦ RETURN 0 AS n1 UNION CALL {FOREACH (...)}
MATCH (y) RETURN y AS n1
UNION CREATE () RETURN 0 AS n1

- ① empty query, $qc = \{\}$, $gs = \{\}$
- ② RETURN 0 AS n1 $\Rightarrow qc = \{n1\}$
- ③ UNION $\Rightarrow$ qc **cleared**
- ④ CALL with recursive subclauses
- ⑤ two FOREACH define $n2, n3$; MERGE adds label A, property u to gs ; after FOREACH ends, $n2, n3$ removed (out of scope)
- ⑥ MATCH (y), RETURN y AS n1 $\Rightarrow qc = \{y, n1\}$

Generation Walkthrough (the Query of Figure 2)

```
RETURN 0 AS n1
UNION CALL {
  FOREACH (n2 IN [] |
    FOREACH (n3 IN [] |
      MERGE (:A {u:0})))
} MATCH (y)
RETURN y AS n1
UNION CREATE ()
RETURN 0 AS n1
```

- 
- ① (empty)
 - ② RETURN 0 AS n1
 - ③ RETURN 0 AS n1 UNION
 - ④ RETURN 0 AS n1 UNION CALL {}
 - ⑤ RETURN 0 AS n1 UNION CALL {FOREACH (...)}
 - ⑥ RETURN 0 AS n1 UNION CALL {FOREACH (...)}
MATCH (y) RETURN y AS n1
 - ⑦ RETURN 0 AS n1 UNION CALL {FOREACH (...)}
MATCH (y) RETURN y AS n1
UNION CREATE () RETURN 0 AS n1

- ① empty query, $qc = \{\}$, $gs = \{\}$
- ② RETURN 0 AS n1 $\Rightarrow qc = \{n1\}$
- ③ UNION $\Rightarrow$ qc **cleared**
- ④ CALL with recursive subclauses
- ⑤ two FOREACH define $n2, n3$; MERGE adds label A, property u to gs ; after FOREACH ends, $n2, n3$ removed (out of scope)
- ⑥ MATCH (y), RETURN y AS n1 $\Rightarrow qc = \{y, n1\}$
- ⑦ UNION clears qc again; CREATE (), RETURN 0 AS n1

Fine-Grained Query Transformation: The Idea

- Do **not** analyze whole-query semantics.
- Instead, transform *fine-grained units* while preserving semantics:
 - **Clause-level**: modify clauses or prepend new clauses (can change graph state / control flow!);
 - **Expression-level**: rewrite expressions inside clauses.
- Only local semantic equivalence must be guaranteed $\Rightarrow$ works for **arbitrary** queries.

Fine-Grained Query Transformation: The Idea

- Do **not** analyze whole-query semantics.
- Instead, transform *fine-grained units* while preserving semantics:
 - **Clause-level**: modify clauses or prepend new clauses (can change graph state / control flow!);
 - **Expression-level**: rewrite expressions inside clauses.
- Only local semantic equivalence must be guaranteed $\Rightarrow$ works for **arbitrary** queries.

Oracle

Run the original and the transformed query on the *same* initial (empty) database:

Results differ $\Rightarrow$ logic bug found.

Fine-Grained Query Transformation: The Idea

- Do **not** analyze whole-query semantics.
- Instead, transform *fine-grained units* while preserving semantics:
 - **Clause-level**: modify clauses or prepend new clauses (can change graph state / control flow!);
 - **Expression-level**: rewrite expressions inside clauses.
- Only local semantic equivalence must be guaranteed $\Rightarrow$ works for **arbitrary** queries.

Oracle

Run the original and the transformed query on the *same* initial (empty) database:

Results differ $\Rightarrow$ logic bug found.

- Generalizes EET (OSDI'24, expression-level only, for SQL): Cypher's read-write clauses enable *clause-level* transformations too.

Clause-Level Transformations: Four Strategies

Type	Original	T.S.	Transformed
Read-Only	C	DC	CALL {MATCH x WHERE false <clauses>} C
		DC	CALL {MATCH <new pattern> <clauses>} C
		IS	OPTIONAL MATCH <new pattern> C
		UC	CALL {WITH * RETURN x AS i} C
		UC	UNWIND <literal> AS i C
	WITH x	UC	WITH x, y AS i
	MATCH x	IS	MATCH x WHERE true
		IS	MATCH <mirror(x)>
Write/Project	C	DC	FOREACH (i IN [] <clauses>)C
		RW	DELETE <deleted var> C
		RW	CREATE x DELETE x C
		RW	SET x += {} C
	REMOVE x	RW	REMOVE x, x
	DELETE x	RW	DELETE x, x
		IS	DETACH DELETE x
Any	C	IS	WITH * C

Clause-Level Transformations: Four Strategies

Type	Original	T.S.	Transformed
Read-Only	C	DC	CALL {MATCH x WHERE false <clauses>} C
		DC	CALL {MATCH <new pattern> <clauses>} C
		IS	OPTIONAL MATCH <new pattern> C
		UC	CALL {WITH * RETURN x AS i} C
		UC	UNWIND <literal> AS i C
	WITH x	UC	WITH x, y AS i
	MATCH x	IS	MATCH x WHERE true
		IS	MATCH <mirror(x)>
Write/Project	C	DC	FOREACH (i IN [] <clauses>)C
		RW	DELETE <deleted var> C
		RW	CREATE x DELETE x C
		RW	SET x += {} C
	REMOVE x	RW	REMOVE x, x
	DELETE x	RW	DELETE x, x
		IS	DETACH DELETE x
Any	C	IS	WITH * C

- **Dead Code (DC):** insert clauses that never execute, e.g. FOREACH (i IN [] | ...) before a write clause, or CALL {MATCH ... WHERE false ...}.

Clause-Level Transformations: Four Strategies

Type	Original	T.S.	Transformed
Read-Only	C	DC	CALL {MATCH x WHERE false <clauses>} C
		DC	CALL {MATCH <new pattern> <clauses>} C
		IS	OPTIONAL MATCH <new pattern> C
		UC	CALL {WITH * RETURN x AS i} C
		UC	UNWIND <literal> AS i C
	WITH x	UC	WITH x, y AS i
	MATCH x	IS	MATCH x WHERE true
		IS	MATCH <mirror(x)>
Write/Project	C	DC	FOREACH (i IN [] <clauses>)C
		RW	DELETE <deleted var> C
		RW	CREATE x DELETE x C
		RW	SET x += {} C
	REMOVE x	RW	REMOVE x, x
	DELETE x	RW	DELETE x, x
		IS	DETACH DELETE x
Any	C	IS	WITH * C

- **Dead Code (DC):** insert clauses that never execute, e.g. FOREACH (i IN [] | ...) before a write clause, or CALL {MATCH ... WHERE false ...}.
- **Unused Context (UC):** define variables never referenced later, e.g. WITH x $\Rightarrow$ WITH x, y AS i.

Clause-Level Transformations: Four Strategies

Type	Original	T.S.	Transformed
Read-Only	C	DC	CALL {MATCH x WHERE false <clauses>} C
		DC	CALL {MATCH <new pattern> <clauses>} C
		IS	OPTIONAL MATCH <new pattern> C
		UC	CALL {WITH * RETURN x AS i} C
		UC	UNWIND <literal> AS i C
	WITH x	UC	WITH x, y AS i
	MATCH x	IS	MATCH x WHERE true
		IS	MATCH <mirror(x)>
Write/Project	C	DC	FOREACH (i IN [] <clauses>)C
		RW	DELETE <deleted var> C
		RW	CREATE x DELETE x C
		RW	SET x += {} C
	REMOVE x	RW	REMOVE x, x
	DELETE x	RW	DELETE x, x
		IS	DETACH DELETE x
Any	C	IS	WITH * C

- **Dead Code (DC):** insert clauses that never execute, e.g. FOREACH (i IN [] | ...) before a write clause, or CALL {MATCH ... WHERE false ...}.
- **Unused Context (UC):** define variables never referenced later, e.g. WITH x $\Rightarrow$ WITH x, y AS i.
- **Redundant Write (RW):** writes with no net effect, e.g. CREATE x DELETE x, SET x += {}, DELETE x $\Rightarrow$ DELETE x, x.

Clause-Level Transformations: Four Strategies

Type	Original	T.S.	Transformed
Read-Only	C	DC	CALL {MATCH x WHERE false <clauses>} C
		DC	CALL {MATCH <new pattern> <clauses>} C
		IS	OPTIONAL MATCH <new pattern> C
		UC	CALL {WITH * RETURN x AS i} C
		UC	UNWIND <literal> AS i C
	WITH x	UC	WITH x, y AS i
	MATCH x	IS	MATCH x WHERE true
		IS	MATCH <mirror(x)>
Write/Project	C	DC	FOREACH (i IN [] <clauses>) C
		RW	DELETE <deleted var> C
		RW	CREATE x DELETE x C
		RW	SET x += {} C
	REMOVE x	RW	REMOVE x, x
	DELETE x	RW	DELETE x, x
		IS	DETACH DELETE x
Any	C	IS	WITH * C

- **Dead Code (DC)**: insert clauses that never execute, e.g. FOREACH (i IN [] | ...) before a write clause, or CALL {MATCH ... WHERE false ...}.
- **Unused Context (UC)**: define variables never referenced later, e.g. WITH x $\Rightarrow$ WITH x, y AS i.
- **Redundant Write (RW)**: writes with no net effect, e.g. CREATE x DELETE x, SET x += {}, DELETE x $\Rightarrow$ DELETE x, x.
- **Inconsequential Supersession (IS)**: equivalent forms, e.g. MATCH x $\Rightarrow$ MATCH x WHERE true, mirrored patterns (a $\rightarrow$ b $\Rightarrow$ b $\leftarrow$ a), insert WITH *.

Redundant Write exposes a Neo4j logic bug (Figure 10 in the paper):

```
// Original: returns 1 (WRONG)  
CREATE ({x:0}) <-[y:A]-()  
CREATE ({x:endNode(y).x})  
RETURN COUNT ({({x:0})}) AS n
```

```
// Transformed: returns 2 (correct)  
CREATE ({x:0}) <-[y:A]-()  
CREATE (z) DELETE z  
CREATE ({x:endNode(y).x})  
RETURN COUNT ({({x:0})}) AS n
```

Redundant Write exposes a Neo4j logic bug (Figure 10 in the paper):

```
// Original: returns 1 (WRONG)
CREATE ({x:0}) <-[y:A]-()
CREATE ({x:endNode(y).x})
RETURN COUNT ({({x:0})}) AS n
```

```
// Transformed: returns 2 (correct)
CREATE ({x:0}) <-[y:A]-()
CREATE (z) DELETE z
CREATE ({x:endNode(y).x})
RETURN COUNT ({({x:0})}) AS n
```

- Inserting `CREATE (z) DELETE z` preserves semantics but breaks the buggy handling of two *consecutive dependent* `CREATE`s.
- Fix required modifying **17 source files** in Neo4j.

Redundant Write exposes a Neo4j logic bug (Figure 10 in the paper):

```
// Original: returns 1 (WRONG)
CREATE ({x:0}) <-[y:A]-()
CREATE ({x:endNode(y).x})
RETURN COUNT ({({x:0})}) AS n
```

```
// Transformed: returns 2 (correct)
CREATE ({x:0}) <-[y:A]-()
CREATE (z) DELETE z
CREATE ({x:endNode(y).x})
RETURN COUNT ({({x:0})}) AS n
```

- Inserting `CREATE (z) DELETE z` preserves semantics but breaks the buggy handling of two *consecutive dependent* `CREATE`s.
- Fix required modifying **17 source files** in Neo4j.

Note

Clause-level transformations can influence query context and database state $\Rightarrow$ force the GDBMS to execute *different internal logic* that must still yield the same result.

Expression-Level Transformations

Type	Initial	T.S.	Transformed
Bool	P	MI	<code>NOT (NOT P)</code>
		MI	<code>P OR false</code>
		MI	<code>P AND true</code>
	<code>P OR Q</code>	MI	<code>NOT ((NOT P)AND (NOT Q))</code>
	<code>P AND Q</code>	MI	<code>NOT ((NOT P)OR (NOT Q))</code>
	true	MI	<code>true OR P'</code>
		MI	<code>P' OR (NOT P')OR (P' IS NULL)</code>
	false	MI	<code>false AND P'</code>
		MI	<code>P' AND (NOT P')AND (P' IS NOT NULL)</code>
	NULL	MI	<code>NULL {XOR = < > = <=} P'</code>
Num	x	MI	<code>-(-x)</code>
		MI	<code>x {+ -} 0</code>
		MI	<code>x {* / ^} 1</code>
	0	MI	<code>0 * x</code>
	NULL	MI	<code>NULL {+ - * /} x</code>
Str	S	MI	<code>S + ""</code>
List	L	MI	<code>L + []</code>
Any	<code>E IS NULL</code>	MI	<code>NOT (E IS NOT NULL)</code>
	E	CF	<code>CASE WHEN true THEN E ELSE E' END</code>
		CF	<code>CASE WHEN false THEN E' ELSE E END</code>

Expression-Level Transformations

Type	Initial	T.S.	Transformed
Bool	P	MI	NOT (NOT P)
		MI	P OR false
		MI	P AND true
	P OR Q	MI	NOT ((NOT P)AND (NOT Q))
	P AND Q	MI	NOT ((NOT P)OR (NOT Q))
	true	MI	true OR P'
		MI	P' OR (NOT P')OR (P' IS NULL)
	false	MI	false AND P'
		MI	P' AND (NOT P')AND (P' IS NOT NULL)
	NULL	MI	NULL {XOR = < < > > = <=} P'
Num	x	MI	-(-x)
		MI	x {+ -} 0
		MI	x {* / ^} 1
	0	MI	0 * x
	NULL	MI	NULL {+ - * /} x
Str	S	MI	S + ""
List	L	MI	L + []
Any	E IS NULL	MI	NOT (E IS NOT NULL)
	E	CF	CASE WHEN true THEN E ELSE E' END
		CF	CASE WHEN false THEN E' ELSE E END

- **Mathematical Identity (MI):** De Morgan's laws, double negation, $x \equiv x + 0$, $S \equiv S + ""$, NULL XOR $P' \equiv \text{NULL}$, ...
- **Cypher Feature (CF):** e.g. $E \equiv \text{CASE WHEN true THEN E ELSE E' END}$ (new components E' built from available variables in qc).

Expression-Level Transformations

Type	Initial	T.S.	Transformed
Bool	P	MI	NOT (NOT P)
		MI	P OR false
		MI	P AND true
	P OR Q	MI	NOT ((NOT P)AND (NOT Q))
	P AND Q	MI	NOT ((NOT P)OR (NOT Q))
	true	MI	true OR P'
		MI	P' OR (NOT P')OR (P' IS NULL)
	false	MI	false AND P'
		MI	P' AND (NOT P')AND (P' IS NOT NULL)
	NULL	MI	NULL {XOR = < < > = <= >= <=>} P'
Num	x	MI	-(-x)
		MI	x {+ -} 0
		MI	x {*/ / ^} 1
	0	MI	0 * x
Str	S	MI	S + ""
List	L	MI	L + []
Any	E IS NULL	MI	NOT (E IS NOT NULL)
	E	CF	CASE WHEN true THEN E ELSE E' END
		CF	CASE WHEN false THEN E' ELSE E END

- **Mathematical Identity (MI):** De Morgan's laws, double negation, $x \equiv x + 0$, $S \equiv S + ""$, $\text{NULL XOR } P' \equiv \text{NULL}$, ...
- **Cypher Feature (CF):** e.g. $E \equiv \text{CASE WHEN true THEN } E \text{ ELSE } E' \text{ END}$ (new components E' built from available variables in qc).

Example (Figure 11 in the paper) — MI reveals a Neo4j planner bug:

```

RETURN NULL AS x                                     //
    Original:      NULL
RETURN NULL XOR EXISTS{()} AS x                       //
    Transformed:  FALSE (WRONG!)
  
```

Root cause: planner operator `LetSelectOrSemiApply` ignored NULL arguments.

Implementation

- Fully automatic framework, $\sim 10\text{k}$ lines of Go; open-sourced.
- Supports **all non-administrative clauses** (21/25 in Neo4j).

Clause	GDsmith	GDBMeter	GraphGenie	GAMERA	Dinkel
MATCH	●	●	●	●	●
CREATE	●	●	●	●	●
MERGE	○	○	○	○	●
DELETE	○	●	○	○	●
REMOVE	○	●	○	○	●
SET	○	●	○	○	●
UNWIND	●	○	●	●	●
WITH	●	○	●	●	●
RETURN	●	●	●	●	●
CALL	○	○	●	○	●
FOREACH	○	○	○	○	●
UNION	○	○	○	○	●
EXISTS	○	○	○	○	●
COUNT	○	○	○	○	●

Implementation

- Fully automatic framework, ~10k lines of Go; open-sourced.
- Supports **all non-administrative clauses** (21/25 in Neo4j).

Clause	GDsmith	GDBMeter	GraphGenie	GAMERA	Dinkel
MATCH	●	●	●	●	●
CREATE	●	●	●	●	●
MERGE	○	○	○	○	●
DELETE	○	●	○	○	●
REMOVE	○	●	○	○	●
SET	○	●	○	○	●
UNWIND	●	○	●	●	●
WITH	●	○	●	●	●
RETURN	●	●	●	●	●
CALL	○	○	●	○	●
FOREACH	○	○	○	○	●
UNION	○	○	○	○	●
EXISTS	○	○	○	○	●
COUNT	○	○	○	○	●

- Bug detection:** logic bugs (result comparison) + internal errors (error messages) + crashes (assertions, ASan for C/C++).
- Query reduction:** first automatic Cypher reducer — delete or replace clauses one by one, restart until fixpoint.

Experimental Setup & Bug Detection (Q1)

Setup: Neo4j (from 5.6.0, incl. Enterprise), RedisGraph (from 2.12.0, later FalkorDB 4.0.0), Memgraph (from 2.7.0); ~9-month intermittent campaign; 64-core AMD EPYC 7742, 256 GB RAM.

Experimental Setup & Bug Detection (Q1)

Setup: Neo4j (from 5.6.0, incl. Enterprise), RedisGraph (from 2.12.0, later FalkorDB 4.0.0), Memgraph (from 2.7.0); ~9-month intermittent campaign; 64-core AMD EPYC 7742, 256 GB RAM.

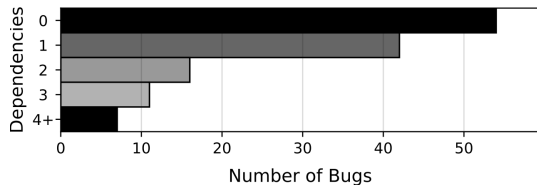
GDBMS	Reported	Confirmed	Fixed
Neo4j	51	51	49
RedisGraph	62	48	29
Memgraph	14	14	6
Total	127	113	84

GDBMS	Logic Bug	Internal Error	Crash
Neo4j	13	38	0
RedisGraph	18	10	34
Memgraph	2	8	4
Total	33	56	38

- **127 unique new bugs:** 51 Neo4j, 62 RedisGraph, 14 Memgraph; **113 confirmed, 84 fixed**, none duplicated.
- **33 logic bugs**, 56 internal errors, 38 crashes (29 memory corruptions + 9 assertion failures).
- 19 bugs only in Neo4j *Enterprise*; Neo4j sent swag as appreciation.

Where and How Are the Bugs Triggered?

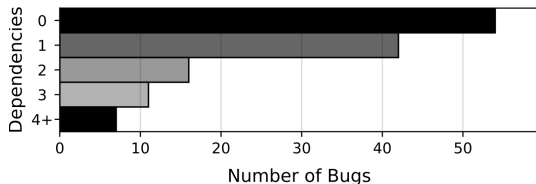
GDBMS	Parser	Planner	Executor
Neo4j	18 (0)	18 (5)	10 (3)
RedisGraph	15 (1)	15 (3)	19 (3)
Total	23 (1)	23 (8)	29 (6)



Where and How Are the Bugs Triggered?

GDBMS	Parser	Planner	Executor
Neo4j	18 (0)	18 (5)	10 (3)
RedisGraph	15 (1)	15 (3)	19 (3)
Total	23 (1)	23 (8)	29 (6)

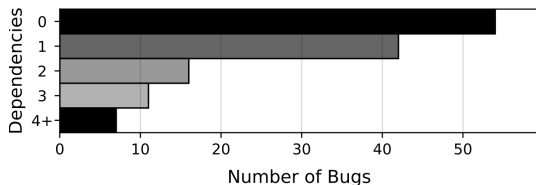
- **Deep logic:** 67.8% (40/59) of analyzed fixed bugs affect the *planner* or *executor*, not just the parser.



Where and How Are the Bugs Triggered?

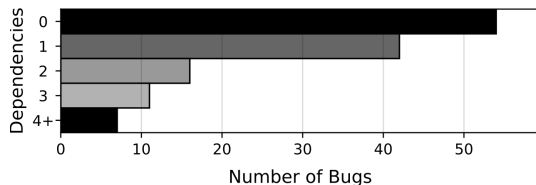
GDBMS	Parser	Planner	Executor
Neo4j	18 (0)	18 (5)	10 (3)
RedisGraph	15 (1)	15 (3)	19 (3)
Total	23 (1)	23 (8)	29 (6)

- **Deep logic:** 67.8% (40/59) of analyzed fixed bugs affect the *planner* or *executor*, not just the parser.
- **Data dependencies matter:** 58% of bug-triggering queries need ≥ 1 dependency (the Figure 1 query has 8).



Where and How Are the Bugs Triggered?

GDBMS	Parser	Planner	Executor
Neo4j	18 (0)	18 (5)	10 (3)
RedisGraph	15 (1)	15 (3)	19 (3)
Total	23 (1)	23 (8)	29 (6)



- **Deep logic:** 67.8% (40/59) of analyzed fixed bugs affect the *planner* or *executor*, not just the parser.
- **Data dependencies matter:** 58% of bug-triggering queries need ≥ 1 dependency (the Figure 1 query has 8).
- **Small after reduction:** 112/127 bugs triggered by queries < 120 bytes (largest: 249 bytes).

Query Quality (Q2) & Ablation (Q3)

GDBMS	Gen.	Valid	Clause	Dep.	Size
Neo4j	318k	89.90%	40.97	26.77	1394.69
RedisGraph	1'078k	93.90%	32.71	30.59	1071.58
Memgraph	570k	83.27%	29.95	21.71	993.63
Average	655k	89.02%	34.54	26.36	1153.3

- 655k queries per GDBMS in 48h; **89.02% valid**; on average **34.5 clauses**, **26.4 dependencies**, 1153 bytes per query.
- Throughput 3.79 tests/s, dominated by GDBMS execution time (74.75% on Neo4j).

Query Quality (Q2) & Ablation (Q3)

GDBMS	Gen.	Valid	Clause	Dep.	Size
Neo4j	318k	89.90%	40.97	26.77	1394.69
RedisGraph	1'078k	93.90%	32.71	30.59	1071.58
Memgraph	570k	83.27%	29.95	21.71	993.63
Average	655k	89.02%	34.54	26.36	1153.3

- 655k queries per GDBMS in 48h; **89.02% valid**; on average **34.5 clauses**, **26.4 dependencies**, 1153 bytes per query.
- Throughput 3.79 tests/s, dominated by GDBMS execution time (74.75% on Neo4j).

Ablation (Table 9 + Table 10 in the paper):

- Without *query context*: **81 bugs** (12 logic) would be missed; without *graph schema*: 14 bugs (3 logic) missed.

Query Quality (Q2) & Ablation (Q3)

GDBMS	Gen.	Valid	Clause	Dep.	Size
Neo4j	318k	89.90%	40.97	26.77	1394.69
RedisGraph	1'078k	93.90%	32.71	30.59	1071.58
Memgraph	570k	83.27%	29.95	21.71	993.63
Average	655k	89.02%	34.54	26.36	1153.3

- 655k queries per GDBMS in 48h; **89.02% valid**; on average **34.5 clauses**, **26.4 dependencies**, 1153 bytes per query.
- Throughput 3.79 tests/s, dominated by GDBMS execution time (74.75% on Neo4j).

Ablation (Table 9 + Table 10 in the paper):

- Without *query context*: **81 bugs** (12 logic) would be missed; without *graph schema*: 14 bugs (3 logic) missed.
- Of 33 logic bugs: **25 by clause-level**, 8 by expression-level transformations; all 6 strategies contribute (IS most: 8, MI least: 3).
- The 25 clause-level bugs show the advantage over expression-only EET.

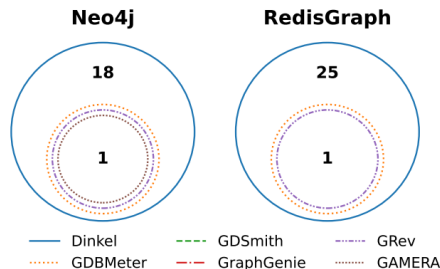
Comparison with the State of the Art (Q4)

Bug latency: 47 bugs existed before 2023 (10 logic bugs) — missed by GDsmith, GDBMeter, GraphGenie, GAMERA, GRev in their extensive evaluations.

Comparison with the State of the Art (Q4)

Bug latency: 47 bugs existed before 2023 (10 logic bugs) — missed by GDsmith, GDBMeter, GraphGenie, GAMERA, GRev in their extensive evaluations.

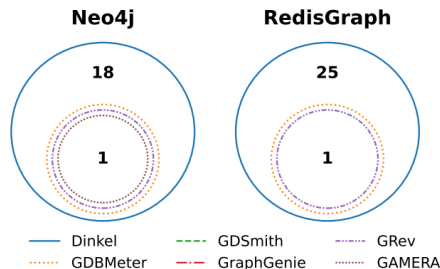
	Neo4j		RedisGraph	
	Line (Avg $\pm$ SD)	Bug	Line (Avg $\pm$ SD)	Bug
DINKEL	750k $\pm$ 5.2k	19	20k $\pm$ 0.1k	26
GDBMeter	283k $\pm$ 0.2k	1	19k $\pm$ 0.3k	1
GRev	597k $\pm$ 4.0k	1	14k $\pm$ 0.1k	1
GAMERA	283k $\pm$ 0.1k	1	3k $\pm$ 0.1k	0
GDsmith	515k $\pm$ 8.6k	0	15k $\pm$ 0.1k	0
GraphGenie	506k $\pm$ 2.0k	0	3k $\pm$ 0.1k	0



Comparison with the State of the Art (Q4)

Bug latency: 47 bugs existed before 2023 (10 logic bugs) — missed by GDsmith, GDBMeter, GraphGenie, GAMERA, GRev in their extensive evaluations.

	Neo4j		RedisGraph	
	Line (Avg $\pm$ SD)	Bug	Line (Avg $\pm$ SD)	Bug
DINKEL	750k $\pm$ 5.2k	19	20k $\pm$ 0.1k	26
GDBMeter	283k $\pm$ 0.2k	1	19k $\pm$ 0.3k	1
GRev	597k $\pm$ 4.0k	1	14k $\pm$ 0.1k	1
GAMERA	283k $\pm$ 0.1k	1	3k $\pm$ 0.1k	0
GDsmith	515k $\pm$ 8.6k	0	15k $\pm$ 0.1k	0
GraphGenie	506k $\pm$ 2.0k	0	3k $\pm$ 0.1k	0



- **Coverage:** +71% lines over existing tools on Neo4j (750k vs. best 597k by GRev), +85% on RedisGraph.
- **Bugs in 48h:** Dinkel 19 (Neo4j) + 26 (RedisGraph); every other tool found ≤ 1 , all subsumed by Dinkel.
- The 43 Dinkel-only bugs need complex dependencies or clauses (FOREACH, UNION, ...) unsupported by existing tools.

Conclusion & Takeaways

Contributions

- **State-aware query generation:** qc + gs abstractions, clause-by-clause construction $\Rightarrow$ complex *and* valid queries.
- **Fine-grained transformation:** clause- and expression-level, semantic-preserving $\Rightarrow$ a *general* oracle for arbitrary queries.
- 127 new bugs (33 logic) in Neo4j / RedisGraph / Memgraph; 84 fixed.

Conclusion & Takeaways

Contributions

- **State-aware query generation:** qc + gs abstractions, clause-by-clause construction $\Rightarrow$ complex *and* valid queries.
- **Fine-grained transformation:** clause- and expression-level, semantic-preserving $\Rightarrow$ a *general* oracle for arbitrary queries.
- 127 new bugs (33 logic) in Neo4j / RedisGraph / Memgraph; 84 fixed.

Discussion & future directions

- Validity gap ($\sim 11\%$): value tracking / constraint solving for illegal arithmetic (e.g., division by zero).
- Purely black-box random generation: coverage feedback could guide it.
- Oracle limits: same-wrong-results bugs, performance and transaction/concurrency bugs are out of scope.
- Extension to GQL / Gremlin and more GDBMSs; richer transformation rules.